

1. Introduction

This report presents the design, implementation, and analysis of four generative deep learning models built for the **Dates Generator** task. The objective is to generate a valid calendar date d that simultaneously satisfies four input conditions: **day-of-week**, **month**, **leap-year flag**, and **decade**.

Each input sample takes the form:

```
[DAY] [MONTH] [LEAP] [DECADE] → dd-mm-yyyy
```

For example, the condition [WED] [JAN] [False] [180] requires a date that falls on a Wednesday in January of a non-leap year in the 1800s (year 1800–1809). The dataset contains 146,462 such (condition, date) pairs spanning year 1800 to 2200.

The four models implemented are:

- **AE** — Conditional Autoencoder (deterministic)
- **VAE** — Variational Autoencoder (stochastic decoder)
- **CVAE** — Conditional Variational Autoencoder (fully conditional)
- **CGAN** — Conditional Generative Adversarial Network (adversarial)

2. Problem Formulation

2.1 Input Representation

Each condition is encoded as a **62-dimensional one-hot vector** formed by concatenating four sub-vectors:

Field	Values	Dimension
Day of week	MON TUE WED THU FRI SAT SUN	7
Month	JAN FEB MAR APR MAY JUN JUL AUG SEP OCT NOV DEC	12
Leap year	False / True	2
Decade	180 ... 220 (41 values)	41
Total		62

The condition vector has exactly four active bits (one per field). This sparse representation keeps the condition signal clear and avoids any implicit ordinal relationship between months or decades.

2.2 Output Representation

Each date is represented as a sequence of **8 digit tokens** in the format `d1 d2 m1 m2 y1 y2 y3 y4` (e.g. 03-12-1962 → [0, 3, 1, 2, 1, 9, 6, 2]). Each token is drawn from the vocabulary {0, ..., 9}. This per-digit decomposition lets every model produce an independent probability distribution over 10 classes for each position, which is easy to train with cross-entropy loss.

2.3 Primary Evaluation Metric

The main metric is the **Condition Satisfaction Rate (CSR)**: the fraction of generated dates that satisfy *all four* conditions simultaneously. Because multiple valid dates exist for most conditions, digit-level accuracy is not meaningful — CSR measures the true generation quality. It is computed live at the end of each training epoch using the validation set.

A secondary check decomposes CSR into per-field accuracy (day, month, leap, decade) to pinpoint which condition a model struggles with most.

3. Shared Infrastructure

3.1 Tokenizer (model/utils/tokenizer.py)

The tokenizer provides two bidirectional mappings:

- **encode_condition(line)** — parses the four bracketed tokens and returns a float32 tensor of shape (62,).
- **encode_date(date_str)** — converts 'dd-mm-yyyy' to a LongTensor of shape (8,).
- **decode_date(digits)** — converts an (8,) tensor back to 'dd-mm-yyyy' for output and validation.

3.2 Dataset (model/utils/dataset.py)

Two PyTorch Dataset classes are provided. **DatesDataset** loads data.txt and returns (condition_vector, digit_sequence) pairs. **ConditionOnlyDataset** loads the inference file (conditions without dates) and returns (raw_line, condition_vector) pairs.

The full dataset (146,462 samples) is split 90 / 10 into training and validation sets using a fixed random seed for reproducibility.

3.3 Validation (model/utils/validation.py)

The **validate(condition_line, date_str)** function checks all four conditions against a Python `datetime.date` object. The function returns `False` for invalid calendar dates (e.g. 30-02-1900) as well. **condition_accuracy(conditions, dates)** wraps `validate` to return the CSR over a batch.

4. Models

4.1 Autoencoder (AE)

4.1.1 Motivation

The Autoencoder is the simplest of the four models. It serves as a **deterministic baseline**: given a (condition, date) pair, it learns to compress and then reconstruct the date. Because there is no stochastic component, its latent space is deterministic and cannot be sampled from freely — making it the weakest generator of the four.

4.1.2 Architecture

Component	Input	Architecture	Output
Encoder	Digit embeddings (8×16=128) + Condition (62)	128+62=190 → Linear(256) → ReLU → Linear(256) → ReLU → Linear(64)	$z \in \mathbb{R}^{64}$
Decoder	z (64) + Condition (62)	64+62=126 → Linear(256) → ReLU → Linear(256) → ReLU → Linear(80)	Logits (8, 10)

The digit embedding maps each of the 10 digit values to a 16-dimensional vector. The encoder concatenates the flattened digit embeddings with the condition vector. The decoder receives the latent z and the condition, producing 80 logits which are reshaped to (8, 10) for 8 independent digit predictions.

4.1.3 Training

Loss function: cross-entropy reconstruction only (no regularisation term).

```
L_AE = CrossEntropy( Decoder(Encoder(x, c), c), x )
```

Optimiser: Adam (lr = 1e-3). Scheduler: StepLR ×0.5 every 10 epochs. Batch size: 512. Epochs: 30.

4.1.4 Inference

At generation time the encoder is bypassed entirely. The latent vector is fixed at $z = \mathbf{0}$ (the learned "centre" of the latent space), and the decoder is called with `Decoder(0, condition)`. This forces the decoder to rely solely on the condition vector to produce a valid date.

4.1.5 Strengths & Limitations

- **Strength:** Fast to train, stable, easy to debug.
- **Strength:** Deterministic output — useful when reproducibility is needed.
- **Limitation:** Cannot generate diverse outputs for the same condition (no sampling).
- **Limitation:** Zero-vector inference is an approximation; the decoder was never trained on $z=0$.

4.2 Variational Autoencoder (VAE)

4.2.1 Motivation

The VAE extends the AE by introducing a **probabilistic latent space**. Rather than mapping to a single point z , the encoder outputs a Gaussian distribution (μ, σ^2) from which z is sampled. The KL divergence regularisation forces this distribution towards the standard normal, enabling true sampling at inference time.

The key architectural decision in this implementation is that the **encoder ignores the condition vector**. Only the decoder sees the condition. This forces the latent space to capture generic date

structure while the decoder handles the condition-specific mapping — a semi-supervised formulation sometimes called a *conditional decoder VAE*.

4.2.2 Architecture

Component	Input	Architecture	Output
Encoder	Digit embeddings (8×16=128) only — NO condition	128 → Linear(256) → ReLU → Linear(256) → ReLU → $\mu, \log\sigma^2$ heads	$\mu, \log\sigma^2 \in \mathbb{R}^{64}$
Reparameterisation	$\mu, \log\sigma^2$	$z = \mu + \sigma \odot \epsilon, \epsilon \sim N(0, I)$	$z \in \mathbb{R}^{64}$
Decoder	z (64) + Condition (62)	126 → Linear(256) → ReLU → Linear(256) → ReLU → Linear(80)	Logits (8, 10)

4.2.3 Training

Loss: Evidence Lower BOund (ELBO) = reconstruction – $\beta \cdot \text{KL}$

$$\begin{aligned} L_{\text{VAE}} &= \text{CrossEntropy}(\text{logits}, x) + \beta \cdot \text{KL}[N(\mu, \sigma^2) \parallel N(0, I)] \\ \text{KL} &= -\frac{1}{2} \cdot \sum (1 + \log\sigma^2 - \mu^2 - \sigma^2) \end{aligned}$$

Optimiser: Adam (lr=1e-3). $\beta=1.0$. StepLR $\times 0.5$ every 10 epochs. Batch: 512. Epochs: 30.

4.2.4 Inference

Sample $z \sim N(0, I)$ then compute `Decoder(z, condition)`. Because the KL term trained the encoder distribution towards the prior, sampling from $N(0, I)$ is valid and produces diverse outputs.

4.2.5 Comparison with AE

Property	AE	VAE
Encoder input	digits + condition	digits only
Latent space	Deterministic point	Gaussian distribution
KL regularisation	None	Yes ($\beta=1.0$)
Inference sampling	$z = 0$ (fixed)	$z \sim N(0, I)$
Output diversity	None (deterministic)	High

4.3 Conditional Variational Autoencoder (CVAE)

4.3.1 Motivation

The CVAE is the direct extension of the VAE in which **both encoder and decoder receive the condition vector**. This allows the posterior distribution $q(z \mid x, c)$ to be condition-aware, which typically results in a tighter, more informative latent space and better generation quality compared to the VAE.

4.3.2 Architecture

Component	Input	Architecture	Output
-----------	-------	--------------	--------

Encoder	Digit embeddings (128) + Condition (62) = 190	$190 \rightarrow \text{Linear}(256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256) \rightarrow \text{ReLU} \rightarrow \mu, \log\sigma^2$	$\mu, \log\sigma^2 \in \mathbb{R}^{64}$
Reparameterisation	$\mu, \log\sigma^2$	$z = \mu + \sigma \odot \varepsilon, \varepsilon \sim N(0, I)$	$z \in \mathbb{R}^{64}$
Decoder	z (64) + Condition (62)	$126 \rightarrow \text{Linear}(256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(80)$	Logits (8, 10)

The only architectural difference from the VAE is in the encoder: the condition vector (62 dims) is concatenated with the digit embeddings (128 dims) before the first linear layer, making the input dimension 190 instead of 128.

4.3.3 Training

Identical ELBO loss to the VAE, but conditioned on c :

$$L_{\text{CVAE}} = \text{CrossEntropy}(\text{logits}, x) + \beta \cdot \text{KL}[q(z|x, c) || N(0, I)]$$

4.3.4 Comparison with VAE

Property	VAE	CVAE
Encoder input	digits only	digits + condition
Posterior	$q(z x)$	$q(z x, c)$
Latent information	Generic date structure	Date structure + condition
Expected CSR	Lower	Higher (tighter posterior)

4.4 Conditional Generative Adversarial Network (CGAN)

4.4.1 Motivation

The CGAN takes a fundamentally different approach. Instead of an encoder-decoder framework, it employs a **two-player adversarial game** between a Generator G and a Discriminator D . G tries to produce dates that D cannot distinguish from real ones, while D tries to correctly classify real vs generated dates. Both networks are conditioned on the input condition vector.

4.4.2 Architecture

Generator

Input	Architecture	Output
Noise z (64) + Condition (62) = 126	$\text{Linear}(256) \rightarrow \text{LeakyReLU}(0.2) \rightarrow \text{BatchNorm} \rightarrow \text{Linear}(256) \rightarrow \text{LeakyReLU}(0.2) \rightarrow \text{BatchNorm} \rightarrow \text{Linear}(80)$	Logits (8, 10)

Discriminator

Input	Architecture	Output
-------	--------------	--------

Date repr (8×10=80) + Condition (62) = 142	Linear(256) → LeakyReLU(0.2) → Dropout(0.3) → Linear(256) → LeakyReLU(0.2) → Dropout(0.3) → Linear(1)	Real/Fake logit (1,)
---	---	-------------------------

The Generator uses **LeakyReLU(0.2)** (standard for GANs to avoid dying neurons) and **BatchNormalization** for training stability. The Discriminator uses **Dropout(0.3)** to prevent it from becoming too strong and starving the Generator of gradient signal.

4.4.3 The Gumbel-Softmax Trick

A critical challenge in GAN training with discrete outputs (digit tokens) is that the argmax operation is not differentiable — the generator gradient cannot flow back through a hard discrete sample. We solve this with the **Gumbel-Softmax relaxation**:

```
soft_sample = GumbelSoftmax(logits, tau=1.0, hard=False)
```

During the Discriminator update, the soft samples are passed as *detached* tensors (no gradient). During the Generator update, gradients flow through the soft samples back to G. Real dates are fed to D as one-hot vectors (the hard discrete equivalent).

4.4.4 Loss Functions

Network	Loss	Formula
Discriminator D	Binary cross-entropy (real=1, fake=0)	$L_D = \frac{1}{2} [\text{BCE}(D(\text{real}, c), 1) + \text{BCE}(D(\text{fake}, c), 0)]$
Generator G	Fool D — maximise $D(G(z, c))$	$L_G = \text{BCE}(D(G(z, c)), 1)$

4.4.5 Training

The Discriminator and Generator are trained in alternating steps within each mini-batch. Separate Adam optimisers are used with different learning rates:

- $lr_G = 2e-4$ — Generator is trained more slowly to avoid mode collapse.
- $lr_D = 1e-4$ — Discriminator updates use a lower rate to stay competitive.
- Batch size: 512. Epochs: 50. Adam $\beta_1=0.5$ (standard for GANs).

4.4.6 Strengths & Limitations

- **Strength:** Can learn complex multi-modal distributions over valid dates.
- **Strength:** No encoder at inference — pure feed-forward generation from noise.
- **Limitation:** Training instability — G and D losses must stay balanced.
- **Limitation:** Mode collapse risk — G may map many noise vectors to the same date.

5. Model Comparison

5.1 Architectural Summary

Property	AE	VAE	CVAE	CGAN
Encoder input	digits+cond	digits only	digits+cond	N/A
Latent type	Deterministic	Gaussian	Conditional Gaussian	Noise $N(0, I)$
KL regularisation	No	Yes	Yes	No
Inference z	$z = 0$	$z \sim N(0, I)$	$z \sim N(0, I)$	$z \sim N(0, I)$
Output diversity	None	High	High	High
Training stability	Very stable	Stable	Stable	Can be unstable
Train epochs	30	30	30	50
Loss terms	Recon only	Recon + KL	Recon + KL	$L_G + L_D$

5.2 Data Flow Diagram

AE: [digits + cond] $\rightarrow$ Encoder $\rightarrow z \rightarrow$ Decoder(z , cond) $\rightarrow$ logits

VAE: [digits] $\rightarrow$ Encoder $\rightarrow (\mu, \sigma) \rightarrow z \sim N(\mu, \sigma) \rightarrow$ Decoder(z , cond) $\rightarrow$ logits

CVAE: [digits+cond] $\rightarrow$ Encoder $\rightarrow (\mu, \sigma) \rightarrow z \sim N(\mu, \sigma) \rightarrow$ Decoder(z , cond) $\rightarrow$ logits

CGAN: [noise+cond] $\rightarrow G \rightarrow$ logits vs real_dates + cond $\rightarrow D \rightarrow$ real/fake

5.3 Expected Performance Ordering

Based on the model designs, the expected CSR ordering from highest to lowest is:

CVAE $\geq$ VAE $>$ CGAN $>$ AE

CVAE is expected to perform best because both encoder and decoder are condition-aware, leading to a tighter, more informative posterior.

VAE is close behind; the stochastic latent space allows diverse sampling, but the encoder's blindness to the condition makes the posterior less sharp.

CGAN can theoretically match or exceed the VAEs, but adversarial training is sensitive to hyperparameters and can suffer from mode collapse.

AE is the weakest generator since it cannot produce diverse outputs. Its CSR depends entirely on how well the decoder has memorised the condition-to-date mapping.

6. Training Setup

Hyperparameter	AE	VAE	CVAE	CGAN
Optimiser	Adam	Adam	Adam	Adam (separate G/D)

Learning rate	1e-3	1e-3	1e-3	G: 2e-4 / D: 1e-4
Adam β_1	0.9	0.9	0.9	0.5
Batch size	512	512	512	512
Epochs	30	30	30	50
LR scheduler	StepLR $\times$ 0.5/10 ep	StepLR $\times$ 0.5/10 ep	StepLR $\times$ 0.5/10 ep	None
Train split	90%	90%	90%	90%
Val split	10%	10%	10%	10%
KL β	—	1.0	1.0	—
Noise dim (CGAN)	—	—	—	64
Latent dim	64	64	64	64
Hidden dim	256	256	256	256

All experiments use **seed = 42** for the dataset split and weight initialisation. Training curves (loss and CSR per epoch) are saved as PNG files and training history is stored in JSON format alongside the model weights.

8. Conclusion

This project implemented four generative models — AE, VAE, CVAE, and CGAN — for the task of generating valid calendar dates from structured conditions. The models progressively incorporate more sophisticated probabilistic machinery:

- **AE** establishes a deterministic baseline that compresses (condition, date) pairs but cannot generate diverse outputs.
- **VAE** adds stochastic sampling through a Gaussian latent space, enabling diversity while keeping the encoder unconditional.
- **CVAE** closes the loop by conditioning both encoder and decoder, giving the model a tighter posterior and the best expected generation quality.
- **CGAN** uses adversarial training to implicitly learn the data distribution without explicit likelihood optimization, offering a complementary approach to the VAE family.

The condition satisfaction rate (CSR) metric is the most meaningful measure of success for this task, since many valid dates exist per condition. The shared infrastructure (tokenizer, dataset, validation) ensures all models are evaluated on an equal footing

Training Results

Model	Epochs	Final Loss	Time
AE	20	~0.000 (overfit)	~3 min
VAE	20	0.1373	~4 min
CVAE	20	0.1402	~4 min
CGAN	10	G=1.05 / D=0.58	~5 min

Evaluation Summary (1000 samples)

Model	All-4 ✓	Day	Month	Leap	Decade
AE	5.4%	14.3%	82.7%	79.8%	35.4%
VAE	9.8%	13.4%	99.1%	77.2%	99.1%
CGAN	10.7%	13.7%	98.8%	77.2%	98.4%
CVAE	11.9%	15.4%	99.2%	74.2%	99.2%